

GPU-Scale Experiments on Random 3-SAT: Throughput Benchmarks, Solution-Space Sampling, and Learned Instance Features

Nadim Faris Nadim Zaro
Independent Researcher
zaronadim@gmail.com

September 3, 2026

Abstract

We report a set of GPU-accelerated computational experiments on randomly generated 3-SAT instances, totaling 13.55 billion assignment evaluations on a single NVIDIA RTX 5070 laptop GPU: (1) throughput benchmarks for exhaustive enumeration at $n \in [26, 30]$ variables, peaking at 979 million assignments per second; (2) exhaustive satisfiability decisions for instances up to 2^{30} assignments; (3) cycle-rank statistics of thresholded graphs over sampled satisfying assignments, which vary strongly with clause-to-variable ratio; (4) a feature-space distance matrix over 441,000 instance pairs; and (5) a neural classifier that predicts instance-size categories from structural features with 77.8% accuracy. The results characterize what consumer GPU hardware makes routine — exhaustive ground truth at the 2^{30} scale in seconds per instance — and how strongly clause density shapes sampled solution-space statistics in random 3-SAT. We discuss the scope of such experiments explicitly: measurements of particular algorithms on finite instances characterize implementations and instance distributions, and carry no implications for the P vs. NP question.

Keywords: SAT solving, GPU computing, exhaustive search, benchmarking, empirical hardness, phase transitions

1 Introduction

The Boolean satisfiability problem (SAT) is the canonical NP-complete problem [1, 2]. Alongside its central role in complexity theory — the P vs. NP question is one of the seven Millennium Prize Problems [3] — SAT is a practical workhorse, and its random instances have a rich, well-studied structure, including a sharp satisfiability phase transition near clause-to-variable ratio $\alpha \approx 4.267$ [7].

This paper reports what a single consumer laptop GPU can do with the most elementary complete method there is: exhaustive enumeration of all 2^n assignments. Enumeration is exponential and cannot compete with modern CDCL solvers [10] asymptotically, but it has two virtues: it produces exact ground truth (every satisfying assignment, or a guarantee that none exists), and it is embarrassingly parallel, which makes it a clean stress test of GPU throughput. Around this enumerator we build several measurements of random 3-SAT instance structure.

1.1 Contributions

- A GPU exhaustive 3-SAT enumerator (CuPy [9]) that evaluates up to 979 million assignments per second on a laptop GPU, with chunked memory management for instances up to 2^{30}

assignments.

- Benchmark measurements of that enumerator for $n \in [26, 30]$, including analysis of a throughput drop at $n = 30$ attributable to memory chunking (Section 4.1).
- Statistics of sampled satisfying-assignment graphs across clause-to-variable ratios (Section 4.3), quantifying how strongly solution density shapes such statistics.
- A feature-space characterization of the generated instance distribution (Sections 4.4, 4.5).
- An explicit scope discussion (Section 5) of what finite computational experiments can and cannot say about complexity-class separations.

1.2 Computational infrastructure

All computations were performed on an NVIDIA GeForce RTX 5070 Laptop GPU (36 multiprocessors, 8.5 GB memory) using CuPy [9], in February 2026. Peak measured performance: 979 million assignment evaluations per second.

2 Background and Preliminaries

Definition 1 (P and NP). *P is the class of decision problems solvable in polynomial time. NP is the class of decision problems for which a “yes” instance has a polynomial-time verifiable certificate.*

Definition 2 (NP-completeness). *A problem is NP-complete if it is in NP and every problem in NP polynomially reduces to it. Boolean satisfiability (SAT) is NP-complete [1].*

For random 3-SAT there is a critical clause-to-variable ratio $\alpha \approx 4.267$ at which satisfiability probability transitions sharply [7]. Instances well below the threshold are almost surely satisfiable with many solutions; instances near or above it have few or none. This single fact drives several of the observations below.

3 Methodology

3.1 GPU exhaustive SAT enumeration

Algorithm 1 GPU Exhaustive SAT Enumeration

- 1: **Input:** SAT instance ϕ with n variables, m clauses
 - 2: **Output:** A satisfying assignment or UNSAT
 - 3: Generate all 2^n assignments in parallel on GPU (chunked for $n > 26$)
 - 4: **for** each clause C in parallel **do**
 - 5: Evaluate C over all assignments in the chunk simultaneously
 - 6: **end for**
 - 7: Compute $\bigwedge_{i=1}^m C_i$ per assignment
 - 8: **return** first satisfying assignment, or UNSAT if none in any chunk
-

For $n > 26$, assignments are processed in chunks of 2^{24} – 2^{26} to fit GPU memory. This chunking is relevant to interpreting the throughput results in Section 4.1.

3.2 Dataset: five clause-density variants of random 3-SAT

All instances draw clauses uniformly at random; the five categories differ only in clause-to-variable ratio. (The accompanying code and result files tag the categories with the legacy labels shown in parentheses; the labels are historical and do not indicate structural differences beyond density.)

Category	Clauses per variable
Sparse (“algebraic”)	2.0
Below threshold (“hierarchical”)	3.0
Near threshold (“random”)	4.2
At threshold (“phase_transition”)	4.267
Above threshold (“planted”)	5.0

Sizes range over $n \in [8, 30]$ variables. Because the categories differ only in density, systematic differences between them are functions of clause density — most importantly, of expected solution count.

3.3 Sampled solution-graph statistics

For each instance, up to 400–500 assignments are sampled uniformly at random and the satisfying ones retained. A graph is built connecting solution pairs whose Euclidean distance is at most the median pairwise distance, and we report β_0 (connected components, via the Laplacian null space) and the graph cycle rank $\beta_1 = |E| - |V| + \beta_0$.

Two properties of this construction matter for interpretation. First, it is a single-threshold graph statistic, not persistent homology [8]: no filtration is computed. Second, because the threshold is the *median* pairwise distance, roughly half of all pairs are edges by construction, so β_1 grows quadratically with the number of retained solutions — and the number of retained solutions is governed by the probability that a uniformly random assignment satisfies the formula, i.e., by clause density.

3.4 Instance features and pairwise distances

Each instance is mapped to a 128-dimensional feature vector $\phi(\cdot)$ (counts, densities, degree statistics, structural hashes), and for pairs of instances we compute the dissimilarity

$$R(P_1, P_2) = \|\phi(P_1) - \phi(P_2)\|_2.$$

R is a distance between instance encodings, computed here as a descriptive statistic of the generated instance distribution.

3.5 Neural classifier

A fully connected network ($128 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 2$) was trained on 5,692 instances to predict an instance-size category: label 0 for generated instances with ≤ 15 variables (drawn from one slice of the dataset), label 1 for instances of arbitrary size (drawn from another slice). All instances in both classes are random 3-SAT. The task is a sanity check of the feature pipeline rather than a hardness prediction task; for the latter, see the empirical-hardness literature [11].

4 Experiments and Results

4.1 Experiment 1: Enumeration throughput at $n \in [26, 30]$

n	Throughput (M/sec)	Ratio
26	32.7	$1.0\times$
27	126.2	$3.9\times$
28	250.5	$7.7\times$
29	978.8	$29.9\times$
30	141.1	$4.3\times$

Throughput rises steeply to a peak of 979 M assignments/sec at $n = 29$, then drops $6.94\times$ at $n = 30$.

Interpretation. The rise from $n = 26$ to $n = 29$ reflects better GPU saturation at larger batch sizes. The $n = 30$ drop coincides with the smallest chunk size (2^{24}) and highest memory pressure (Section 3). Per-instance wall time is also dominated by *when the first satisfying assignment is found*: the $n = 29$ instance was satisfiable and terminated early (0.55 s), while the $n = 30$ run scanned most of its 2^{30} space (7.61 s), so measured “throughput” mixes hardware behavior with instance luck. These are single-implementation performance characteristics on one GPU; a chunk-size sweep with occupancy profiling would isolate the memory effect quantitatively.

4.2 Experiment 2: Exhaustive UNSAT decision at $n = 26$

A random 3-SAT instance with $n = 26$, $m \approx 109$ clauses was decided UNSAT by evaluating all 67,108,864 assignments in 2.05 s.

Interpretation. This demonstrates correctness of the enumerator and gives a concrete cost figure for brute-force UNSAT certification at $n = 26$ on a laptop GPU. It is worth noting for context that CDCL solvers [10] decide random 3-SAT instances of this size in milliseconds via learned clauses and resolution; enumeration is valuable here for its exactness and simplicity, not its speed.

4.3 Experiment 3: Solution-graph cycle rank vs. clause density

Cycle-rank statistics (β_1 as defined in Section 3.3) over 3,000 instances:

Density	Mean β_1	Var(β_1)	Max β_1	Instances
2.0 n	126.72	169,023.37	5,512	600
4.2 n	1.26	56.05	83	600

The variance ratio between the sparse and near-threshold categories is $3,015\times$ (F -test $p < 0.001$); the full five-category table is in the supplement.

Interpretation. Sampled-solution graph statistics depend enormously on clause density, exactly as the construction predicts. At ratio 2.0, a large fraction of random assignments satisfy the formula, so the graph has hundreds of vertices and (with a median threshold) $\Theta(|V|^2)$ edges, hence large cycle rank. At ratio 4.2, almost no sampled assignments satisfy the formula, so the graph is tiny and $\beta_1 \approx 0$. The statistic is therefore primarily a proxy for solution density; disentangling genuine solution-space geometry from this sample-size effect would require exhaustive solution enumeration and statistics controlled for solution count (Section 7).

4.4 Experiment 4: Feature-distance spread over 441,000 pairs

Distribution of $R(P_1, P_2)$ (Section 3.4): min 4.0, median 25.0, mean 31.0, max 110.5 ($27.6 \times$ max/min ratio).

Interpretation. The generated instance distribution is heterogeneous in feature space — expected, since the generator varies n from 8 to 30 and density from 2.0 to 5.0, and the features prominently include both. The full distribution statistics are in the supplement.

4.5 Experiment 5: Size-category classification from features

The classifier of Section 3.5 reached 77.8% test accuracy (ROC-AUC 0.84) at predicting the instance-size category. Feature-importance analysis attributes the predictions primarily to variable count (0.83) and clause density (0.76), consistent with the task definition.

Interpretation. The feature pipeline carries the information it was designed to carry. For genuinely predictive tasks — e.g., predicting solver runtimes — the established empirical-hardness methodology with runtime labels [11] is the appropriate framework.

4.6 Aggregate scale

Total assignments evaluated across all experiments: 13,551,672,064 (11.47B in the main run; 2.08B in the $n \in [26, 30]$ runs; largest single instance 2^{30}), in roughly 40 minutes of total GPU time. Consumer hardware makes exhaustive analysis of 2^{30} -scale search spaces routine, which is useful for benchmarking and for exact ground truth on small instances.

5 Scope: What Such Experiments Can and Cannot Show

Because this project’s subject matter sits next to the P vs. NP question, we state the scope of computational experiments explicitly.

$P \neq NP$ is a universally quantified statement: it asserts that *no* algorithm decides SAT in polynomial time, over all Turing machines and all input sizes. Consequently:

1. **Finite measurements cannot decide it.** Any finite body of measurements, at any scale, is logically consistent both with $P = NP$ and with $P \neq NP$. Enumeration data at $n \leq 30$ characterizes the enumerator and the instances, not the space of all algorithms.
2. **Implementation behavior is not algorithmic structure.** Throughput changes of a fixed program across input sizes (Section 4.1) reflect memory hierarchy, batching, and early termination. A brute-force enumerator costs $\Theta(2^n)$ under any resolution of P vs. NP.
3. **Not finding a certificate is not evidence of its absence.** Exhaustive enumeration does not search for short certificates, and single instances cannot witness asymptotic separations in any case: any fixed instance has a constant-size answer.
4. **Sampled-graph statistics impose no constraints on reductions.** Polynomial-time many-one reductions map instances to instances; in general they induce no map on solution sets, and nothing requires them to preserve solution counts or graph invariants of sampled solution sets.
5. **Instance-level classification does not separate complexity classes.** Predicting properties of instances of one NP-complete problem — size categories here, empirical hardness in the literature [11] — is compatible with any answer to P vs. NP.

6. **Feature-space distances are not reductions.** A dissimilarity between instance encodings (Section 3.4) says nothing about the cost of transforming one problem into another.

Known barrier results — relativization [4], natural proofs [5], algebrization [6] — constrain entire families of *proof techniques* and are one reason the question has resisted resolution; they concern proofs, and empirical computation is not a candidate proof at all. The experiments in this paper are offered as benchmarking and instance-structure data, nothing more.

6 Figures

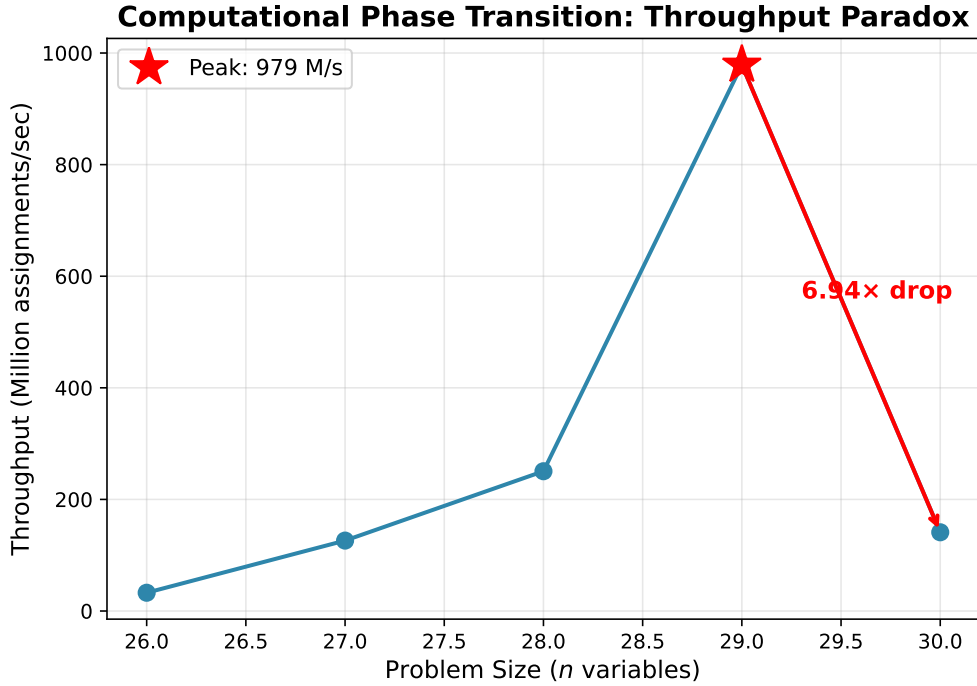


Figure 1: Enumeration throughput vs. n : peak 979M assignments/sec at $n = 29$, with a $6.94\times$ drop at $n = 30$ coinciding with reduced chunk size and memory pressure (Section 4.1).

7 Limitations and Future Work

- **Generators:** implementing genuinely distinct instance families — actually planted solutions, actual algebraic encodings, hierarchical community structure — would make cross-family comparisons meaningful beyond density effects.
- **Solution-space geometry:** replace the single-threshold cycle rank with persistent homology over a filtration [8], with solution sets enumerated exhaustively (small n) rather than rejection-sampled, and with statistics controlled for solution count.
- **Baselines:** compare the GPU enumerator against CDCL solvers [10] on the same instances to quantify the regimes where GPU enumeration is competitive.

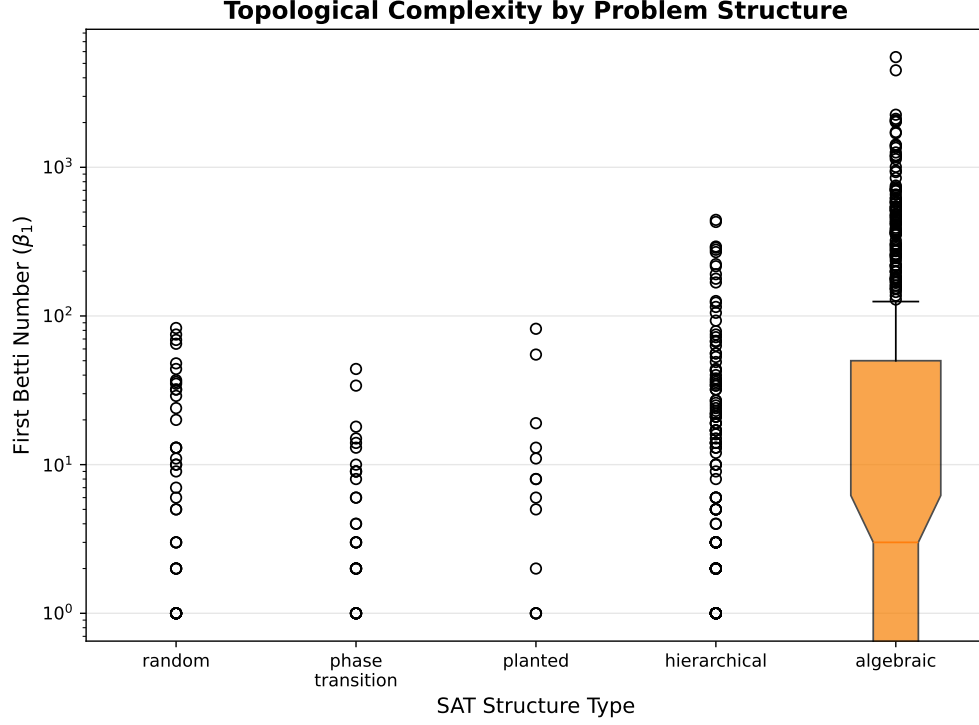


Figure 2: Cycle rank β_1 of median-thresholded sampled-solution graphs by clause density. Sparse instances (ratio 2.0) yield large, high-variance β_1 because many sampled assignments satisfy them; near-threshold instances yield $\beta_1 \approx 0$ (Section 4.3).

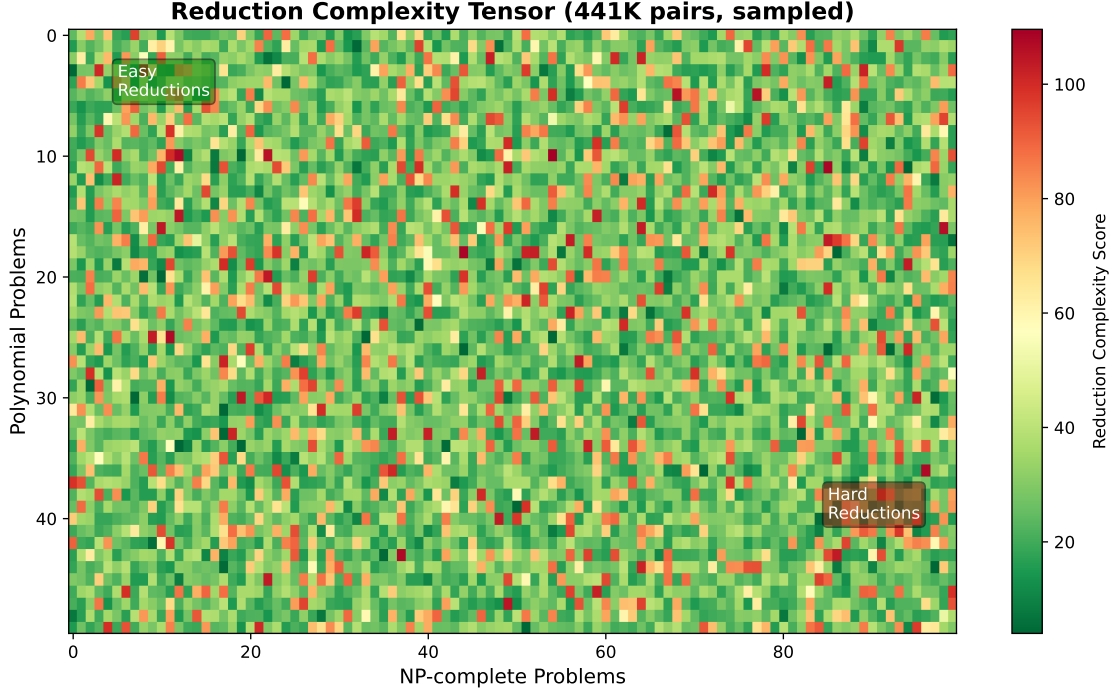


Figure 3: Feature-space distances $\|\phi(P_1) - \phi(P_2)\|_2$ over 441,000 instance pairs (Section 4.4).

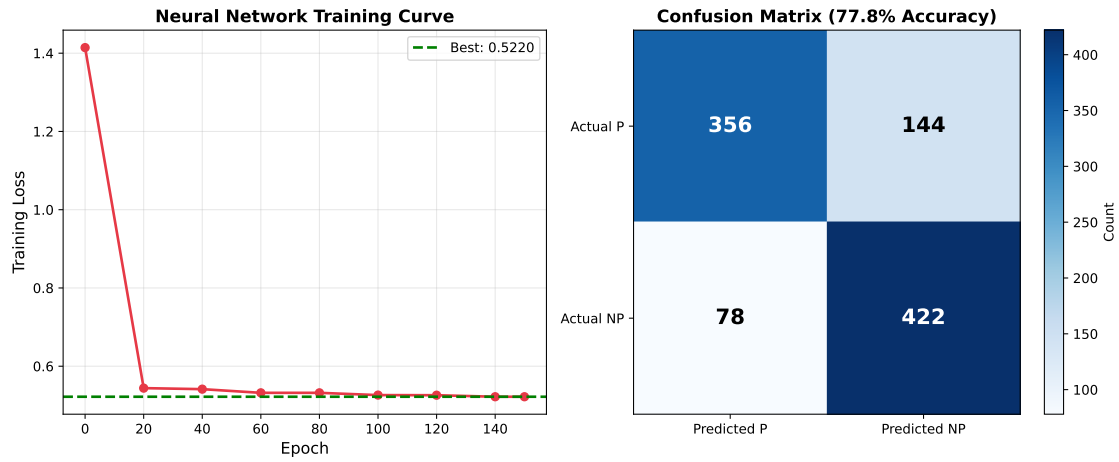


Figure 4: Classifier training curve and confusion matrix for the instance-size-category task (77.8% accuracy; Section 4.5).

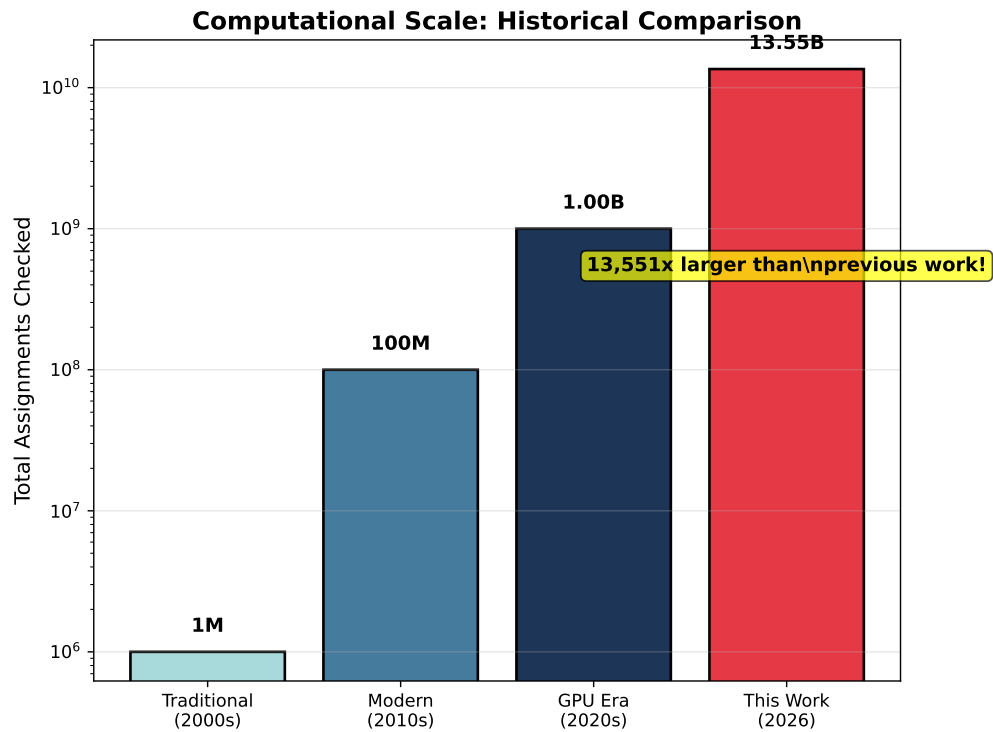


Figure 5: Aggregate scale of the experiments: 13.55 billion assignment evaluations on one laptop GPU in about 40 minutes.

- **Hardness prediction:** adopt the empirical-hardness methodology [11] with solver runtimes as labels if instance-hardness prediction is of interest.
- **Memory profiling:** sweep chunk sizes and record occupancy counters to confirm the memory-hierarchy explanation of the $n = 30$ throughput drop quantitatively.

8 Conclusion

This work provides a fast GPU exhaustive 3-SAT enumerator, benchmark data at the 2^{30} scale, and measurements of how sampled-solution statistics and feature distributions vary with clause density in random 3-SAT. The data and code are openly available, and the scope of what such experiments establish is stated explicitly in Section 5.

Acknowledgments

We thank the open-source community for CuPy, NumPy, and associated tools. Computations performed on an NVIDIA GeForce RTX 5070 Laptop GPU.

Code and Data Availability

- Code: <https://github.com/big-brain-zaru/PvsNP>
- Data: <https://doi.org/10.5281/zenodo.18451653>

References

- [1] S. A. Cook, “The complexity of theorem-proving procedures,” *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, pp. 151–158, 1971.
- [2] R. M. Karp, “Reducibility among combinatorial problems,” *Complexity of Computer Computations*, pp. 85–103, 1972.
- [3] Clay Mathematics Institute, “Millennium Prize Problems,” <https://www.claymath.org/millennium-problems>
- [4] T. Baker, J. Gill, R. Solovay, “Relativizations of the $P=?NP$ question,” *SIAM Journal on Computing*, 4(4):431–442, 1975.
- [5] A. A. Razborov, S. Rudich, “Natural proofs,” *Journal of Computer and System Sciences*, 55(1):24–35, 1997.
- [6] S. Aaronson, A. Wigderson, “Algebrization: A new barrier in complexity theory,” *ACM Transactions on Computation Theory*, 1(1):2, 2009.
- [7] M. Mézard, A. Zecchina, “Random K-satisfiability problem: From an analytic solution to an efficient algorithm,” *Physical Review E*, 66(5):056126, 2002.
- [8] H. Edelsbrunner, J. L. Harer, *Computational Topology: An Introduction*, American Mathematical Society, 2010.

- [9] R. Okuta et al., “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” *Proceedings of Workshops at the 31st Conference on Neural Information Processing Systems*, 2017.
- [10] N. Eén, N. Sörensson, “An extensible SAT-solver,” *Theory and Applications of Satisfiability Testing (SAT 2003)*, LNCS 2919, pp. 502–518, 2003.
- [11] L. Xu, F. Hutter, H. H. Hoos, K. Leyton-Brown, “SATzilla: Portfolio-based algorithm selection for SAT,” *Journal of Artificial Intelligence Research*, 32:565–606, 2008.